

DEVOPS

- By Santhosh Kumar

CICD PIPELINES

CI/CD stands for Continuous Integration and Continuous Deployment or Continuous Delivery. It is a process used in DevOps to automate software development, testing, and deployment. CI/CD pipelines help developers release applications quickly and with fewer errors.

Continuous Integration means developers regularly add their code changes into a shared repository like Git. Whenever new code is added, automatic build and testing processes start. This helps identify bugs early.

Continuous Deployment means after testing is completed successfully, the application is automatically deployed to servers or production environments. This reduces manual work and speeds up software delivery.

In many companies, CI/CD pipelines are created using tools like Jenkins, GitHub Actions, GitLab CI/CD, Azure DevOps Pipelines, Docker, and Kubernetes.

Real-time example:

Suppose developers are working on an online shopping application. One developer adds a payment feature and another developer adds a delivery tracking feature. Both developers push their code into GitHub. The CI/CD pipeline automatically builds the application, runs test cases, checks for errors, and deploys the updated application to the testing or production server.

CI/CD pipelines improve software quality because testing happens automatically. It also saves time because deployment is automated. Companies like Amazon, Netflix, and Microsoft use CI/CD pipelines to release updates regularly without affecting users.

Advantages of CI/CD pipelines:

- Faster software delivery
- Automatic testing and deployment
- Reduced manual work

DEVOPS

- By Santhosh Kumar

- Early bug detection
- Better software quality
- Faster feedback for developers
- Improved teamwork between developers and operations teams

Simple CI/CD Flow:

- Developer writes code
- Code pushed to Git repository
- Build process starts
- Testing runs automatically
- Application deployed to server
- Monitoring starts after deployment

HOW APPLICATION OR PRODUCT GOES LIVE

In software companies, applications do not directly move from development to production. Before going live, the application passes through different stages and testing processes to ensure stability and quality.

Initially, developers write code and test it in their local systems. After development is completed, the code is pushed into repositories like GitHub or Azure Repos. The CI/CD pipeline automatically builds the application and performs testing.

Once testing is completed successfully, the application is deployed to different environments such as Development, QA, UAT, and Production. After approvals and validations are completed, the product goes live for customers.

Real-time example:

Suppose a company develops a food delivery application. Developers first create login and ordering features. Testers then check whether the features work correctly. After successful testing, the application is

DEVOPS

- By Santhosh Kumar

deployed to staging and production servers. Finally, customers can install and use the application from the Play Store or website.

Steps followed before application goes live:

- Requirement gathering
- Designing
- Development
- Code review
- Build creation
- Testing
- Deployment to environments
- User acceptance testing
- Production deployment
- Monitoring and maintenance

During deployment, DevOps engineers monitor servers, application performance, database connectivity, and security. If any issue occurs after release, teams quickly fix the issue using hotfixes or rollback methods.

Companies use deployment strategies like:

- Blue-Green Deployment
- Rolling Deployment
- Canary Deployment

These methods help reduce downtime and avoid application failures during release.

DIFFERENT ENVIRONMENTS WHILE WORKING ON A PROJECT

In software development, different environments are used to develop, test, and deploy applications safely. Each environment has a separate purpose.

DEVOPS

- By Santhosh Kumar

Development Environment

This is the environment where developers write and test code initially. Developers use this environment to create new features and fix bugs.

Real-time example:

A developer creates a payment feature for an e-commerce application and tests it in the development environment before sharing it with testers.

Testing or QA Environment

QA stands for Quality Assurance. In this environment, testers verify whether the application works correctly and identify bugs.

Testing types include:

- Functional Testing
- Regression Testing
- Integration Testing
- Performance Testing

Real-time example:

Testers check whether payment transactions, login systems, and order placement features work properly in the QA environment.

UAT Environment

UAT stands for User Acceptance Testing. Business users or clients test the application in this environment before production release.

They verify whether the application satisfies business requirements.

Real-time example:

The client checks whether the shopping application works according to customer expectations before giving approval for release.

Staging Environment

The staging environment is almost similar to the production environment. Final testing is done here before the application goes live.

DEVOPS

- By Santhosh Kumar

This environment helps identify production-related issues before actual deployment.

Real-time example:

The DevOps team checks server configurations, database connections, APIs, and application performance in staging before deployment.

Production Environment

Production is the live environment where real users access the application.

This environment should be stable, secure, and highly available because customers use it directly.

Real-time example:

Amazon website used by customers for shopping is the production environment.

Importance of Multiple Environments

Using multiple environments helps companies:

- Reduce risks
- Identify bugs early
- Improve software quality
- Avoid production failures
- Perform testing safely
- Ensure smooth deployment

In modern software companies, DevOps and CI/CD pipelines automate movement between these environments. This helps applications move faster from development to production with better reliability and fewer errors.